

Finding one in fifty thousand

A multi-turn shopping agent that hunts a hidden product through conversation. Sixteen features, six days, and an 8.5× score — with every dead end written down.

HITRATE@10	MRR	URNS TO FIND	TECHNICALSCORE
98.0%	0.864	2.85	0.9122
from 12.5% 7.8×	from 0.068 12.7×	from 9.81 3.4× faster	from 0.1067 8.5×

Regenerated 2026-08-31 against commit `dff52e8` · branch `dev` · deadline 2026-09-01 12:00 +08

HOW TO READ THIS

Sections 1–3 are written for anyone — no code, no jargon that isn't explained on the spot. Sections 4–7 carry the engineering detail. Section 8 onward is the honest accounting: what failed, what we chose not to do, and what is left. Every number here is measured and reproducible, never estimated.

SECTION 01

The one-page summary

What we built, where it landed, and the three things worth knowing about how it got there.

What we built

A conversational shopping assistant. A customer arrives with a vague idea — *"I'm looking for a winter jacket"* — and somewhere in a catalogue of 50,000 products there is one specific item they have in mind. They will not tell us which. The agent has **ten turns** to work it out: it can ask one clarifying question per turn, and it shows a ranked list of its ten best guesses. The moment the right product appears anywhere in that list, the session ends and we are scored on how high up it landed and how long it took.

Think of it as Twenty Questions — except the answer is one item out of fifty thousand, we only get ten questions, and we are marked on how confidently we point at the right one.

Where we landed

We find the target in 196 of 200 test conversations, usually within three turns, and when we find it we put it in first place 162 times out of 196.

The three things worth knowing

1. **The biggest win was learning to have a conversation, not building a better search engine.** Feature 03 — asking a question every turn and remembering the answers — took the score from 0.124 to 0.682 in a single change. That is more than every other feature in the project combined.
2. **We deliberately make the agent slower to make it more accurate.** Because the session freezes the instant the target appears, showing a full list early *locks in* a mediocre position. So on turns 1–2 the agent shows only its single best guess, holding the rest back to buy another round of questions. It costs a sliver of the efficiency score and bought **+0.20 of MRR** — a trade the official weighting invites, worked out as arithmetic before a line was written.
3. **We proved when to stop.** Reading the competition's own simulator established that the customer only ever knows **four things**, and is out of information by turn 3. That turned "we seem to have plateaued" into a mathematical certainty, and told us no amount of cleverer questioning could ever help again. Everything after went into ranking quality, robustness and the submission itself — not into chasing an impossible ceiling.

AND ONE THING WE DON'T OVERCLAIM

The last 0.005 of the score rests on **five conversations out of 200**, one of which is clinging to 10th place. We report 0.912205 as *0.907 plus a marginal rescue*, not as a comfortable margin — and this document says so wherever the number appears.

SECTION 02

What the competition actually asks for

The setup, the four kinds of customer, and the rules that shape every decision below.

Each session, the agent is handed an anonymised customer profile and an opening message. It replies with any combination of: a message, **one** clarifying question, and a ranked list of product IDs. The simulated customer answers, and the loop repeats.

- **Catalogue:** 50,000 frozen products — Amazon Reviews 2023, *Clothing, Shoes & Jewelry*.
- **Our test set:** 200 labelled conversations we can run locally, as often as we like.
- **The real exam:** 800 private conversations the organiser keeps hidden until judging.
- **What counts:** only the product ID, matched by exact string equality. Nothing else scores.

Four kinds of customer

The mix is identical in our local set and the hidden one, so per-scenario results are meaningful.

SCENARIO	SHARE	WHAT MAKES IT HARD
buying	40%	Opens with a firm requirement. Should be easy — but a mis-read requirement becomes a filter that hides the answer.
browsing	40%	Opens with almost nothing. Everything depends on the questions we ask.
intent_override	15%	On turn 3 or 4 the customer changes their mind. Nothing found before that moment counts at all.
boundary	5%	The customer refuses to answer — exactly once — then behaves normally. Mistake that refusal for "no more information" and you throw the session away.

The rules we work under

- Only our own agent code is editable. The evaluator, the data and the organiser's specifications are read-only.
- The agent must never read the answer key, even though it sits in the same file the evaluator loads.
- A crash, a malformed reply or a timeout is scored as a **complete miss**. Failing softly is worth more than being clever.
- No API keys in the repository, and judging may run with **no network access at all** — so anything online needs a working offline path.

SECTION 03

The scoreboard, in plain English

Three numbers, one formula, and the two properties of the test set that make every claim in this document checkable.

```
TechnicalScore = 0.50 × HitRate@10 + 0.30 × MRR + 0.20 × Efficiency
Efficiency      = clip((11 - MTTC) / 10, 0, 1)
```

TERM	PLAIN MEANING	WEIGHT
HitRate@10	<i>Did we find it at all?</i> The share of conversations where the target appeared in our top ten within ten turns.	50%
MRR	<i>How near the top?</i> 1st place scores 1.0, 2nd scores 0.5, 5th scores 0.2, a miss scores 0.	30%
MTTC	<i>How fast?</i> Average turn on which we first found it. A miss is charged as turn 11 — the penalty for never finding it.	20%

The weighting is also the priority order: **find it at all** → **rank it near the top** → **get there in fewer turns**. We optimised in exactly that sequence, and the score history in the next section shows it.

TWO PROPERTIES THAT SHAPE EVERYTHING

Our local set is 200 conversations, so **one conversation moves HitRate@10 by 0.005**, and any TechnicalScore change under about **0.01 is noise**. And the runs are **deterministic** — identical code always produces an identical score, so a change in the number is always caused by our change, never by luck. That second property is the foundation of every measurement in this document.

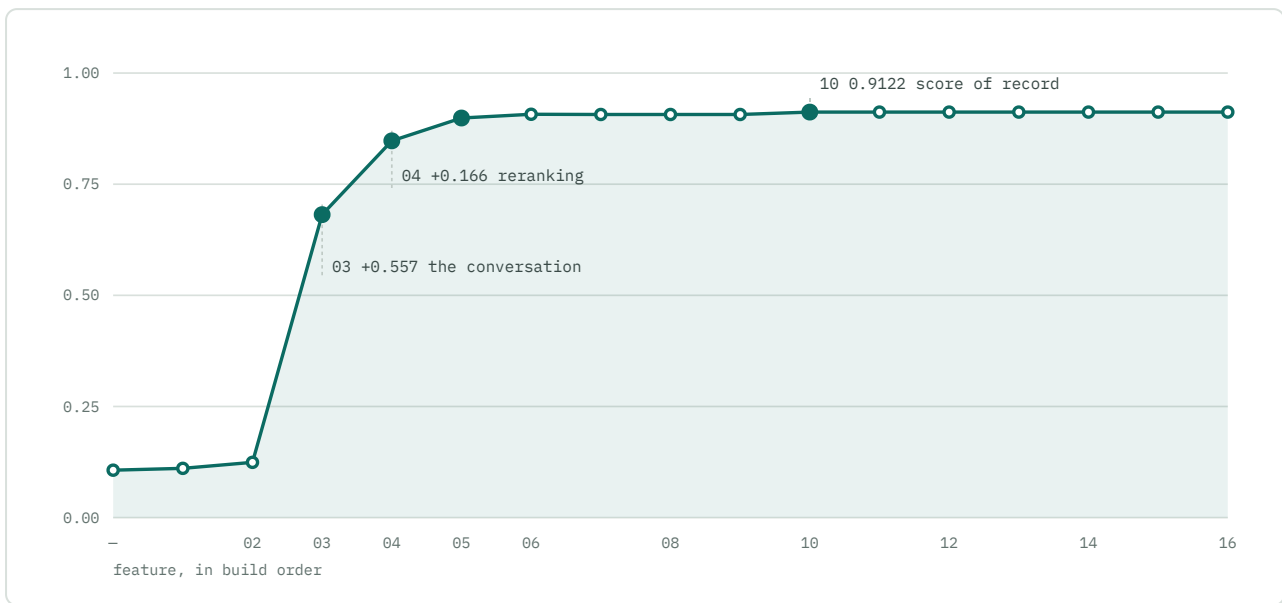
SECTION 04

The journey: 0.107 to 0.912 in five acts

Every point below is a real measured run, taken from a committed results snapshot. Features are numbered in build order.

TechnicalScore after each feature

Hover any point for its full metrics. The two vertical jumps are features 03 and 04.



#	FEATURE	HIT	MRR	TURNS	SCORE	MOVE
—	<i>starter kit</i>	0.125	0.068	9.81	0.10671	—
01	Dual-track intent routing	0.130	0.070	9.76	0.110829	+0.0041
02	Multi-route retrieval	0.150	0.068	9.56	0.124334	+0.0135
03	Clarification loop	0.825	0.420	3.85	0.681542	+0.5572
04	Semantic reranking	0.965	0.652	2.53	0.847520	+0.1660
05	Rank-vs-turn arbitrage	0.965	0.852	2.97	0.898866	+0.0513
06	Phrase retrieval + 2 bug fixes	0.975	0.858	2.88	0.907281	+0.0084
07	Hybrid/dense retrieval	0.975	0.857	2.90	0.906791	−0.0005
08	Latency & token disclosure	0.975	0.857	2.90	0.906791	0
09	Optimization headroom survey	0.975	0.857	2.90	0.906791	0
10	Field-factor calibration	0.980	0.864	2.85	0.912205	+0.0054
11	Free-form input robustness	0.980	0.864	2.85	0.912205	0 · identical
12	Intent override, properly	0.980	0.864	2.85	0.912205	0 · identical
13	Optional language model	0.980	0.864	2.85	0.912205	0 · identical
14	Model circuit breaker	0.980	0.864	2.85	0.912205	0 · identical
15	Free-form negation	0.980	0.864	2.85	0.912205	0 · identical
16	Generic attribute facets	0.980	0.864	2.85	0.912205	0 · identical

"BYTE-IDENTICAL" IS STRONGER THAN "FLAT"

It means the entire 38,523-byte results file — every one of the 200 sessions, not just the headline average — is unchanged down to the byte. Where this document says a feature cost nothing, that is what was verified.

ACT I Laying the plumbing

0.107 → 0.124 · features 01-02 · **+0.018, below the noise floor**

The starter agent was **completely stateless**: every turn it took the incoming message, ran one keyword search, and forgot everything. A customer who said "*a key requirement is: cotton*" on turn 1 got no better answer on turn 5.

Features 01 and 02 fixed the plumbing — per-session memory for detected constraints, and two search routes instead of one, blended together. Combined they moved the score by barely more than noise, and both feature documents say so.

The honest verdict: Act I did not move the number, it made Act II possible. There was nowhere to *put* the answer to a question until the state existed.

ACT II The breakthrough — have a conversation

0.124 → 0.682 · feature 03 · +0.5572, the largest single move in the project

This is the most important change in the project, and it is embarrassingly simple in hindsight. Two bugs, one root cause: **the agent was not treating the conversation as a source of information.**

1. It **never asked anything** — the clarifying-question field was hardcoded to null. In this simulator, a null question makes the customer reply *"ask me about one specific attribute"* and reveal nothing. We were burning all ten turns learning nothing.
2. It **threw away what it was told** — the search ran on *this turn's* text only, so turn 5's query no longer contained the product category from turn 1.

Fix both and the score goes from 0.124 to 0.682. Every scenario improved. The two that were near-zero — browsing (0.038) and boundary (0.000) — became the two strongest, exactly the expected shape: they had the most to gain from a conversation because they start with nothing.

An ablation confirmed both halves earn their place: asking alone gets 0.527, asking plus remembering gets 0.682. Remembering is worthless without asking, which is why they were built and measured as one feature.

The customer's answers are near-verbatim text from the target product's own catalogue entry. Getting the customer to talk is effectively getting them to quote the answer.

So we ask a question every single turn — recommendations are scored every turn regardless, so a question is free.

ACT III **Ordering, not finding**

0.682 → 0.899 · features 04-05 · **+0.217**

After Act II we were *finding* the target 82.5% of the time but MRR was only 0.420 — the signature of finding things and then burying them mid-list.

Feature 04, semantic reranking, replaced "whatever the search engine returned" with a proper second pass: the search produces a pool of 120 candidates and a reranker decides the order, scoring each on how much of the customer's own distinctive phrasing it contains — with rare wording counting for far more than common wording. Unusually, *every* metric improved in *every* scenario, because better ordering also converts misses into hits and converts them earlier.

Feature 05, rank-vs-turn arbitrage, is the most counter-intuitive thing we built, and it came from reading the evaluator rather than from tuning:

The session ends the instant the target appears anywhere in the top 10, and its rank is frozen there. There is no later turn in which to improve it.

So surfacing the target at 8th place on turn 2 permanently banks a poor score — while eight turns of clarification sit unused. The official weights make the trade lopsided:

PER SESSION	VALUE
one extra turn of delay costs	0.0001
one unit of reciprocal rank gains	0.0015

Delay pays for itself if it improves the rank by even one place from 4th; promoting 2nd to 1st is worth *seven* turns of waiting. So the agent shows one recommendation on turns 1–2, four on turn 3, eight on turn 4, and the full list thereafter. The safety catch matters: the moment no questions remain, everything is released — withholding is a bet that better evidence is coming. Across twenty schedule variants, **HitRate never moved**. The gate never costs a find; it only converts turns into rank.

ACT IV Diminishing returns, measured honestly

0.899 → 0.912 · features 06-10 · +0.013, and this is where the discipline shows

Feature 06 was scoped as "build dense retrieval" and became something better: instrumentation revealed the remaining misses were a **recall** failure, not a ranking one — the targets were never retrieved at all, not even 200 deep, so no reranker could have helped. Three distinct causes were found, of which the best is worth telling:

*A regex meant to catch budgets was reading **measurements** as prices. The phrase "...fits up to 8-inch wrist circumference" set a hard filter of **price** ≤ \$8, excluding the very watch the customer was describing. Across the whole test set that filter had fired three times, all wrong, and zero times correctly.*

Feature 07 built genuine semantic retrieval — and measured it as **flat**. We shipped it anyway, at the one configuration verified to cost nothing, as insurance for the hidden set. Blending it into the reranker regressed at every weight tested, so it ships *disabled* with the measurements recorded so nobody re-attempts it on a hunch.

Feature 09 asked "is there anything left?" and answered it properly: 32 configuration variants across 7 axes, of which **exactly one** beat the shipped setup. That one became **feature 10** — the reranker had been *discounting* matches in the two product fields the simulator draws every customer disclosure from.

READ THIS BEFORE QUOTING 0.912205

The +0.005 rests on five sessions: one rescued miss scraping in at rank 10 on turn 5, plus four rank improvements against compensating drift elsewhere (12 sessions improved, 11 worsened). **The justification for shipping it is the mechanism, not the magnitude** — the simulator generates every disclosure from those two fields, and that is equally true of the hidden set.

ACT V Building for humans and for judges

0.912205, held exactly · features 11-16 · every one byte-identical

That is a design constraint, not a coincidence. The scored path and the human path are **disjoint**: a full 200-session run makes 566 calls into the simulator-handling code and **zero** into the free-form human-handling code, because every simulated reply is claimed by an earlier pattern match.

So every capability a live demo needs — understanding someone typing real prose, treating "*not fully polyester*" as an exclusion rather than a requirement, gender and neckline awareness, an optional language model — enters as an optional parameter that is **empty on every scored turn**.

That property is not asserted, it is **enforced**: an automated check asserts the free-form branch runs zero times during a scored run, so if a future edit ever widens a pattern enough to change the score, the check goes red *before* it can happen quietly.

SECTION 05

Feature dossiers, 01–16

One entry per feature, in build order: what it does in plain terms, the technical substance, and the lesson worth keeping. Full write-ups live in `docs/features/`.

01 Dual-track intent routing

TIER 1

RETRIEVAL & ROUTING

0B95776

0.10671 → 0.110829

IN PLAIN TERMS Teach the agent to tell two kinds of shopper apart. Someone who says "I need cotton" should get a *narrower* set of results; someone who says "show me something nice" should get a *wider* one. Before this, both got the same treatment.

TECHNICALLY Per-session state for three slots — price ceiling, colour, material — scraped by regex and remembered across turns. If any slot is filled the session is on the **buying** track: colour and material become required search terms and the price becomes a numeric filter. Otherwise it stays **browsing**: a wide, unfiltered query. Search-column weights were tuned in the same commit.

RESULT +0.005 HitRate is exactly one session out of 200, and +0.004 sits below the noise floor. Recorded as *"laid the plumbing, did not itself move the number."*

LESSON A feature whose value is enabling the next feature is still worth shipping — but it should be reported as such, not dressed up as a win.

02 Multi-route retrieval pipeline

TIER 1

RETRIEVAL & ROUTING

9AFF528

+0.0135

IN PLAIN TERMS Search the catalogue several different ways at once and combine the answers, instead of trusting a single search.

TECHNICALLY A second route restricts the same terms to the product's **categories** field only, so a strong category signal competes on its own merits instead of being diluted by noisy title and description text. The routes merge by **weighted Reciprocal Rank Fusion** — each contributes $\text{weight} / (60 + \text{rank})$ — because their raw scores are not on comparable scales, only their *rankings* are. A backfill net tops the list up if hard filters narrow it too far.

LESSON, AND A WARNING MRR *fell* while HitRate rose. Fusion surfaced targets the keyword route missed entirely, but landed them low while nudging well-ranked targets down. The weights make it a net win — but this is the clearest early example of the two metrics moving in opposite directions, and it is why every feature document since carries a per-scenario table rather than an aggregate alone.

03 Clarification loop and cross-turn evidence

TIER 1

DIALOG + RANKING

+0.5572 – LARGEST MOVE

IN PLAIN TERMS Start asking questions, and start remembering the answers.

TECHNICALLY Four decisions, each drawn from the evaluator's own behaviour:

- **Ask every turn.** Recommendations are scored every turn regardless, so a question costs nothing. There is no ask-versus-recommend trade-off to balance — do both, always.
- **Lead with `other`.** The disclosure filter is `attribute == "other" or classify_constraint(value) == attribute`, so **"other" is the only question that cannot whiff** — it matches any undisclosed constraint and returns up to two per turn. Asking "colour" about a target with no colour constraint burns the turn entirely.
- **Retire spent attributes, but not deflections.** *"I don't have an **additional** preference for X"* means X is empty — retire it. *"I don't have a preference for X"* is the boundary customer deferring once, and must **not** retire it, or we throw away every remaining question in exactly the scenario scoring 0.0.
- **Accumulate evidence.** Disclosures append oldest-first and the query is built from all of them. The opener carries the product category, which a latest-message-only query discards.

LESSON Read the evaluator. The largest win in the project came from understanding the rules of the game, not from a better algorithm.

OUR LARGEST DISCLOSED RISK

The `other` catch-all is a property of the *released* simulator. The specification warns the organiser may paraphrase customer messages, and the private simulator could treat it more strictly. Our question-order fallback degrades to targeted asking rather than breaking, but this remains the single biggest risk to the headline number.

04 Semantic reranking

TIER 2

DIALOG + RANKING

169AB9B

+0.1660

IN PLAIN TERMS Stop letting the search engine decide the final order. Take its best 120 guesses and re-order them on a smarter criterion: *does this product speak the customer's own words, especially the unusual ones?*

TECHNICALLY Two evenly-weighted signals — **coverage** (the share of the evidence's rarity-weighted mass present, discounted by which field matched) and **phrase** (the share surviving as an *intact sequence*). "closure type buckle" appearing verbatim is far stronger evidence than those three words scattered across a page of marketing copy. The whole call is fault-isolated: a reranker failure costs ordering, never the session.

THE PART WE BUILT AND DELETED The first version blended the search engine's own order back in, reasoning that reranking should correct rather than replace. The sweep said otherwise — prior weight 0.35 scored 0.810, 0.10 scored 0.828, **0.00 scored 0.845**. Fusion still chooses *which* 120 candidates are considered and breaks ties; it just no longer votes on the order.

05 Rank-vs-turn arbitrage (deferred disclosure)

TIER 2

DIALOG + RANKING

544838B

+0.0513

IN PLAIN TERMS Show fewer results early, on purpose. Because the session ends the moment we get it right, showing a long list too early locks in a mediocre position when we still had eight turns of questions in hand.

TECHNICALLY A single gate — (1, 1, 4, 8, 10) indexed by turn. The agent still retrieves and reranks a full top ten, then truncates to what the turn has earned. The release valve is the important part: if no attributes remain to ask about, the full list goes out immediately.

RESULT MRR 0.652 → 0.852 for 0.435 of a turn. 65 sessions improved, 134 unchanged, 1 regressed. Rank-1 hits went 106 → 160 of 193; the rank 5–10 tail collapsed from 37 sessions to 10. **The miss set is byte-identical** before and after.

ON THE EFFICIENCY REGRESSION MTTC got *worse* and that is the mechanism working, not a side effect. Trading 0.0435 of efficiency for 0.200 of MRR is exactly the arbitrage the weights invite. Read the two rows together or it looks like a mixed result.

CHOOSING THE SCHEDULE Twenty variants across two rounds; the top four land within 0.0008 of each other. We chose the **least aggressive schedule that still reaches the plateau**, betting least on the simulator's willingness to keep answering. It is also the argmax, so principle and measurement agree rather than having to be traded off.

06 Phrase retrieval, and two constraint bugs

TIER 2

RETRIEVAL & ROUTING

26D4B21

+0.0084

IN PLAIN TERMS Scoped as "build semantic search". Instrumentation showed that was the wrong diagnosis — the missing products were never being retrieved *at all*, so no amount of clever re-ordering could have helped. Three real bugs were found instead.

CAUSE 1 — MEASUREMENTS READ AS BUDGETS *"...fits up to 8-inch wrist circumference"* became a hard `price ≤ 8` filter, excluding the watch being described. Fixed with a unit blocklist plus a digit guard — and the guard is subtle: without it the engine backtracks, `\d+` gives up "30" and retries "3", the unit check then sees "0mm" — which starts with no unit — and *"up to 30mm"* yields a \$3 ceiling.

CAUSE 2 — THE WRONG TOKENIZER The reranker's tokenizer drops stopwords, which is correct there and wrong for querying, because the index still contains them. "Pull On closure" became the query "pull closure" (1 document) when the index holds "pull on closure" (7,184). A common phrase silently became a rare one and matched the wrong products. Two tokenizers now exist deliberately, each documented with why they must not be merged.

CAUSE 3 — INTACT PHRASES WERE NEVER SEARCHED One target held three phrases occurring in exactly **one** product each — three unique fingerprints — and was still never retrieved, because the keyword route dissolves every disclosure into a bag of terms where a distinctive phrase is just five more words in a sixty-term OR. Added a third route querying intact phrases, deliberately **unfiltered**: an intact phrase is stronger evidence than a scraped colour, so a wrong filter must never suppress the one route that identifies the product.

ON THE FLAT DELTA +0.0084 is inside the noise floor and the delta tool flags it. Kept because the movement is not spread thinly — it is entirely inside `buying`, precisely where the mechanism predicts it, and two of the bugs are live on any dataset containing a real price or a stopword phrase.

07 Hybrid / dense retrieval

TIER 2

RETRIEVAL & ROUTING

1D41DEE

FLAT, SHIPPED ANYWAY

IN PLAIN TERMS Real semantic search — matching on *meaning* rather than exact words, so "something warm for hiking" can find a fleece that never uses those words. It did not improve the local score. We shipped it anyway, and the reasoning is the interesting part.

TECHNICALLY TF-IDF into Truncated SVD, fitted from the catalogue's own text at startup — fully offline, no pretrained model, deterministic given a fixed seed. Zero-norm rows are guarded at *construction* time, not just query time, because an unguarded one corrupts every future similarity check against it for the life of the process.

CONFIGURATION	HITRATE	MRR	SCORE
baseline, no dense	0.975	0.857935	0.907281
route only, weight 0.3 (shipped)	0.975	0.857304	0.906791
route 0.5, no blend	0.970	0.856579	0.903374
route 0.3 + reranker blend 0.03	0.975	0.850427	0.905128
route 0.3 + reranker blend 0.1	0.965	0.834117	0.893735

The blend regresses **monotonically the instant it is nonzero** — the same signature the removed positional blend showed in feature 04. The explanation: this technique is a smoothed compression of the same word-frequency statistics the coverage score already reads, so adding it perturbs an already-settled ranking rather than contributing new information.

WHY SHIP A FLAT FEATURE The value it hedges — robustness to paraphrasing — exists only on the hidden set and cannot be measured locally by construction. It ships at the one configuration verified to cost nothing, with the reranker blend disabled but intact and documented, so nobody re-enables it on a hunch.

08 Latency and token-usage disclosure

TIER 3

INTEGRATION

1CFFD0A

0 BY DESIGN

IN PLAIN TERMS The competition requires us to disclose how fast the agent is, how many tokens it uses and what it costs. This measures those honestly rather than asserting them.

TECHNICALLY Timing is recorded in a `finally` block so a turn that *raises* is still timed — an unrecorded timeout is exactly the latency worth knowing about. The 95th percentile is nearest-rank rather than interpolated, because with a few hundred samples interpolation invents a number between two turns that were never observed.

THE NON-OBVIOUS CONSTRAINT

Latency is deliberately **not** returned in the response. The API contract sets `additionalProperties: false`, so an extra `latency_ms` key would be *malformed output* — which the specification says may be scored as a miss. Disclosing latency inside the response would literally cost HitRate. It is read out of the process afterwards instead, with the constraint documented inline so nobody helpfully adds the field later.

ENFORCED, NOT ASSERTED The reporting tool exits with an error if token usage is ever nonzero, so if anyone later adds a model call the "\$0.00" claim fails loudly instead of going quietly stale.

09 Optimization headroom survey

TIER 2

COORDINATION + EVALUATION

INVESTIGATION, NO CODE

IN PLAIN TERMS A deliberate stop-and-check: *is there anything left to optimise, or is the agent finished?* Several features had asserted a plateau and the assertion had hardened into folklore. This re-checked it.

THE FINDING THAT MATTERS The evaluator builds the customer's entire knowledge as at most **four constraints, total, for the whole session**, and returns at most two per turn. Since "other" matches any constraint, **two questions drain the customer completely**:

TURN	EVIDENCE IN HAND
1	the opening message (category)
2	+ 2 constraints
3	+ the remaining 2 — everything obtainable, ever
4+	<i>"I don't have an additional preference"</i> — nothing new arrives

This converts a measured plateau into a **structural proof**, and permanently rules out three directions: deferring further buys zero new evidence; a better question order cannot extract a fifth constraint that does not exist; and personalisation adds nothing the four constraints don't already say. **Given complete evidence by turn 3, the only remaining lever in the entire system is ranking quality.**

WHERE THE POINTS ARE Of 195 hits, only 161 were at rank 1 — 34 landed at ranks 2–8, and *all 34 hit on turn 3 or 4*, exactly where the disclosure gate widens. Turns 1–2 were 84/84 at rank 1.

POOL	WORTH IN TECHNICALSCORE
34 hits at rank 2–8 promoted to rank 1	+0.0348
4 remaining misses converted to finds	+0.0160

The rank pool is worth **over twice** the entire miss pool, and the misses are separately established as unreachable. *Chase rank, not recall.*

ALSO ESTABLISHED The organiser's own difficulty labels are uninformative — easy, medium and hard all sit at 80–81% rank-1 share, so our failures are catalogue

ambiguity, not their hard cases. A suspected truncation bug was killed by measurement (0 of 574 turns exceed the cap). And of 32 variants across 7 axes, **exactly one** beat the shipped configuration; three of the other outside-noise results were confirmations that a shipped value is load-bearing.

10 Field-factor calibration

TIER 2

DIALOG + RANKING

+0.0054 — SCORE OF RECORD

IN PLAIN TERMS The reranker was quietly *penalising* matches found in the two places the customer's own words actually come from. One constant, changed to match the mechanism.

TECHNICALLY The reranker discounts a match by which product field it was found in. Those weights were set by hand in feature 04 and never swept — and they were wrong in a checkable way: the customer's disclosures are generated **verbatim from the features and details fields**, so every word the customer will ever say originates there — and the reranker was discounting them to 0.85 while rewarding the title at 1.0. Raised both to parity.

VARIANT	SCORE	HITRATE	MRR
shipped before	0.906791	0.9750	0.857304
features=1.0 only	0.911124	0.9800	0.860413
details=1.0 only	0.908416	0.9750	0.863054
both = 1.0 (shipped)	0.912205	0.9800	0.864018
both = 1.0, categories = 1.0	0.899816	0.9700	0.846387
both = 1.15 (past title)	0.904549	0.9700	0.856163

Three things this establishes: **features** carries most of the gain, exactly as the mechanism predicts; **1.0 is a ceiling, not a trend** — pushing past parity *loses* score; and it is specific to these two fields, since raising **categories** alongside costs 0.007 and flat-1.0-everywhere costs 0.011. Field discrimination still matters; it was miscalibrated on exactly the two fields carrying the evidence.

THE HONEST DECOMPOSITION One rescued miss landing at rank 10 on turn 5 supplies the entire HitRate gain; the MRR gain comes from four sessions against compensating drift (12 improved, 11 worsened; rank-1 count moved only 161 → 162). **The justification is the mechanism, not the magnitude.** One genuine bright spot: boundary MRR became a perfect 1.0.

11 Free-form input robustness

TIER 3

DIALOG + RANKING

BYTE-IDENTICAL

IN PLAIN TERMS Reported from manual testing: *"agent gets stuck and keeps repeating itself after a colour is mentioned, and changing the colour doesn't work either."* Everything built so far was tuned to the sentence shapes the *simulated* customer emits. A real person types none of them.

TECHNICALLY Four symptoms, one cause — a person's text matched no pattern and was discarded whole, so the question never rotated, the message never changed, no evidence accumulated, and first-write-wins meant a new colour was detected correctly and then silently dropped while the stale one kept filtering. A new final branch handles anything the four scripted patterns don't claim.

THE HALF THAT IS EASY TO MISS The superseded value is scrubbed from the **accumulated evidence, not just the slot**. Replacing the slot alone changes the filter while the ranking keeps serving blue items — which to the user is still "changing the colour doesn't work".

A CORRECTION WORTH RECORDING The first version required an explicit cue (*actually, instead*) before a filled slot could be replaced, reasoning that a passing mention shouldn't overwrite. That is wrong in the case that matters most: asked *"any particular colour?"*, a person answers **"red"** — no cue — so the slot kept **blue** and the original complaint persisted. There is no reading of "red" that means "still blue". The cue gate was removed.

A LESSON ABOUT TESTING

The first version's check asserted *"no two turns produce the same message"*. Because the question rotates every turn, that passes even when the colour never changes — the strings differ in their second half. The reported bug was invisible to it. The check now asserts what the user actually cares about: the colour named must be the last colour typed, and no other colour may appear. **When a test can pass while the reported symptom is still present, the test is measuring the wrong thing.**

12 Intent override, properly

TIER 2

DIALOG + RANKING

D87DEB4

BYTE-IDENTICAL + 2 REJECTIONS

IN PLAIN TERMS When the customer says *"actually, ignore what I said before"*, what should we forget? The intuitive answer — delete the withdrawn preference — is measurably **wrong**, and this feature is mostly the record of proving that.

THE STRUCTURAL FINDING The evaluator builds an override from `old_value = soft[-1]` and `new_value = hard[0]` — **both drawn from the target product's own intent card**. The "abandoned" preference is a true attribute of the very product we are hunting, and the target never changes. **The override is cosmetic**. Acting on the retraction can therefore only destroy true evidence — and since the hidden set is generated by the same function, that holds there too.

VARIANT	WHAT IT DOES	RESULT
A	clear colour + material	flat, byte-identical; fires in 21/30
B — shipped	also clear the price ceiling	flat, byte-identical; fires in 0/30 here
C — rejected	delete the retracted claim from evidence and phrases	−0.003875 (override MRR 0.880 → 0.800)
D — rejected	drop it from phrase routes only	flat, but discards 19 identifying routes

THE INSTRUCTIVE REJECTION Variant D measured **flat and byte-identical** — and instrumenting it showed it was not inert at all. It threw away phrase routes with a document frequency of **one**, each of which puts the target at rank 1 of a one-item list. On the hidden set, a session whose only identifying evidence is such a phrase would turn from a hit into a miss. **Flat on the public set is not the same as safe**.

CLOSED QUESTION Intent override sits at 29/30 with 25 at rank 1 and MTTC 3.69 against a structural floor of 3.60 — 0.09 of a turn left in the whole scenario. The four non-rank-1 sessions are a *ranking* problem: one sits at rank 4 on every single turn, before and after the override.

13 Optional language model (off by default)

TIER 3

INTEGRATION

D3DD020 · F506810

BYTE-IDENTICAL

IN PLAIN TERMS Add the option of a hosted AI model, wired so it can be demonstrated without ever putting the score at risk. The judged configuration makes **no model call at all**.

THE REQUIREMENT WAS EXPLICIT The feature only counts if the score does not go down. That rules out any design where a model sits on the scored path and we argue afterwards that it probably helped.

PROVIDER HISTORY, WHICH IS THE USEFUL PART SiliconFlow was chosen first — but its free tier requires real-name verification assuming mainland-Chinese documents, so no key was ever obtainable. The client was always plain OpenAI-compatible chat completions, so moving to **OpenRouter** changed only the defaults and the docs. The model was then chosen by running a benchmark twice with identical results:

MODEL	PARSE / SLOTS / PRICE / TERMS	LATENCY
inclusionai/ling-3.0-flash-fin:free	100 / 100 / 100 / 100	~1.5 s
nvidia/nemotron-3-super-120b-a12b:free	80 / 100 / 100 / 50	4–11 s
nvidia/nemotron-3.5-lightning:free	0 / 0 / 0 / 0	~8.7 s
others	20/0/0/0 or rate-limited	–

This is the benchmark paying for itself. An interim default had been picked by *reading model cards* and turned out to be unavailable in practice. The measured winner is a **finance**-tuned variant nobody would have guessed — recorded as a caveat rather than hidden, and the first thing to re-measure if quality looks off.

MODE	REACH	SCORE RISK
off (default)	nothing runs	none — byte-identical
freeform	only the human-input branch, unreachable while scoring	none by construction
expand	adds a retrieval route at weight 0.25	real but tiny; unmeasured against a live model

SAFETY Enabling requires **both** a key and an explicit mode — neither alone does anything, so a stray key in a teammate's shell cannot silently change a scored run — and an unrecognised mode fails *closed*. Model output is treated as untrusted input throughout: colours and materials must be single tokens (a multi-word hallucination would become a required search term and empty the catalogue); prices must be positive finite numbers with `bool` explicitly excluded; keywords are charset- and length-clamped, which drops injection-shaped strings outright.

MEASURED Default `off`: byte-identical. Configured for `expand` **with every socket raising** — i.e. judging with the network cut while credentials sit in the shell: **also byte-identical, full document**. The fallback requirement discharged as a measurement rather than an assertion. Against deterministic stubs, the worst case — a model emitting pure noise into the query every turn — costs **0.00013**, about 1/75th of the noise floor, with HitRate untouched.

TWO BUGS FOUND ON THE WAY A modal dialog that could not be closed, because `.modal { display: flex }` is an *author* rule and beats the browser's own `[hidden]` rule regardless of specificity. And an error message that said "the call failed" for a dead network, a rejected key and a spent quota alike — three problems whose fixes have nothing in common.

14 Model circuit breaker

TIER 3

INTEGRATION

BYTE-IDENTICAL

IN PLAIN TERMS If the model endpoint is dead, notice once and stop calling it — rather than waiting for a timeout on every single turn.

TECHNICALLY Three trip conditions, because "unusable" has three shapes: 2 consecutive *connection* failures, 3 consecutive failures of any kind, or 3 consecutive *successes* slower than 4.5 seconds. Once latched, calls return the same "no answer" a failure would, with no socket and no wait, so every existing fallback runs unchanged.

An HTTP error is deliberately **not** classed as a network failure — reaching the service and being told "no" is a service problem, so it takes the slower three-strike path. The breaker **never self-closes**: an automatic retry would put the timeout back on the very turn the breaker exists to protect.

IT FIRED IN ANGER, TWICE, UNPROMPTED During the model benchmark — one model tripped the slow-call rule at ~8.7 s, a rate-limited one tripped the failure rule after three fast 429s. First live confirmations that it works outside a stub.

15 Free-form negation and slot-aware questioning

TIER 3

DIALOG + RANKING

AFA3168

BYTE-IDENTICAL

IN PLAIN TERMS Two defects in a single manual transcript. A person typed "*i want it to be grey and not fully polyester*" — and the agent replied "**Narrowed to items matching grey, polyester.**" It stated the exact opposite of the requirement back to the customer, then filtered and ranked on it. Separately, having been told the colour unprompted, it went on to ask for the colour anyway.

TECHNICALLY Negation is detected by a cue that must end within 24 characters of the value *and* inside the same clause, so "*anything but leather*" negates while "*no rush, I want a black leather belt*" does not. Ruled-out values are recorded **before** any slot write and before the optional model is consulted, so neither a regex nor an AI expansion can reintroduce a refused value.

EXCLUSIONS DEMOTE, THEY DO NOT DELETE Ruled-out products are pushed to the back after reranking — because catalogue material text is noisy ("polyester lining"), a short list is worse than a badly ordered one, and a wrong exclusion must never be able to hide the target.

A SUBTLE DISTINCTION The scrub that removes a negated word is *gentler* than the one that removes a superseded one. "*grey but not polyester*" is one phrase making two claims, and the harsher scrub would have discarded the colour they wanted along with the material they didn't.

ON THE GUARD RAILS Two assertions in the verification harnesses pinned the *old* behaviour and were **updated, not weakened** — a filled colour slot now correctly means its question is spent. Six new checks were added. Design choice recorded: false negatives were chosen over false exclusions, since wrongly excluding is the more damaging error.

16 Generic attribute facets, and a ratchet that enforces the guarantee

TIER 3

DIALOG + RANKING

18785DA

BYTE-IDENTICAL

IN PLAIN TERMS Reported from manual testing: asking for **men's** clothing returned ten women's products. Three distinct defects, all diagnosed before any code changed.

1 – FILLER WORDS DOMINATED THE RANKING **under 50 dollars** was parsed into a price filter *and left in the query as text*:

TERM	DOCUMENTS	RANKING WEIGHT
dollars	56	6.79
tshirt	441	4.73
under	1,743	3.36
men	14,908	1.21

under 50 dollars carried 33% of the entire ranking signal on words describing no product, and **dollars** alone outweighed **tshirt**. Roughly 62% of the mass was noise.

2 – GENDER WAS AN ORDINARY KEYWORD The *weakest* term in the query, in a catalogue where 32,347 of 50,000 products mention "women". A hard requirement for "men" does **not** fix this, which is the non-obvious part: 5,900 products contain "men" outside their title — keyword spam like *"gifts for men women teens"* — so women's listings satisfy the filter. Measured: requiring **men** still returned women's items at ranks 1, 2 and 5. What works is scoping to the **title** and demoting titles asserting a sibling value.

TECHNICALLY A **facet** is a group of mutually exclusive values, and stating one implies rejecting its siblings. Ten groups ship — gender, neckline, sleeve, fit, rise, length, closure, pattern, occasion, season. **Adding one is a dictionary entry, not new code.**

QUERY	BEFORE	AFTER
round neck, blue, cotton, under 50 dollars, men tshirt	0/10 men's	8/10 men's
the agent's reply	"blue, cotton, under \$50.00"	"men, crew neck, blue, cotton, under \$50.00"

TWO GUARANTEES ADDED IN THE SAME COMMIT The **isolation invariant** asserts the free-form branch is called **zero** times across a full scored run, so a future edit that widens a pattern goes red before it can silently change the score. And **the score ratchet** runs the full 200 sessions and exits non-zero if the score fell, distinguishing *byte-identical* from merely *score-equal* — offsetting session movements can hide a regression the 800-session hidden set would not forgive.

SECTION 06

The architecture as it stands

Eight modules, roughly 2,700 lines of agent code, and no network call in the default configuration.

starter/agent.py orchestration + the official reset()/respond() contract	261
starter/retrieval.py search index, query routes, rank fusion	576
starter/dialog_state.py per-session slots, evidence, question policy	560
starter/ranking.py the reranker: coverage + phrase scoring	167
starter/dense_retrieval.py offline semantic embeddings, no file I/O	101
starter/llm.py OPTIONAL hosted model client; off by default, stdlib only	573
starter/facets.py mutually-exclusive attribute groups; human path only	190
starter/env_file.py .env scaffolding; NEVER imported by the scored path	255

What happens in a single turn

STEP 1 – UNDERSTAND

Match the message, extract, remember

Match against the four simulator sentence shapes; scrape colour, material and price ceiling; append the disclosure to accumulated evidence. On the human path only: negation, corrections and facets.



STEP 2 – RETRIEVE**Up to six routes over 50,000 products**

Routes 3–6 are deliberately **unfiltered**, so a wrong filter cannot suppress the one route that identifies the product.

1 keyword whole catalogue	1.0	2 category categories column	0.3	3 phrase up to 12 exact phrases	≤ 0.5
4 dense semantic embeddings	0.3	5 expand model keywords — OFF	0.25	6 facet title-scoped attributes	0.3

**STEP 3 – FUSE****Weighted Reciprocal Rank Fusion → a pool of 120**

Each route contributes $\text{weight} / (60 + \text{rank})$. Backfill tops the pool up if hard filters narrowed it too far.

**STEP 4 – RERANK****50% coverage + 50% intact phrase**

Fusion no longer votes on order; it chose the pool and breaks ties. Excluded terms and opposite facets are demoted afterwards, so the reranker cannot undo them.

**STEP 5 – DISCLOSE****1 · 1 · 4 · 8 · 10 by turn**

Unless no questions remain — then everything is released immediately, because withholding is only ever a bet that better evidence is coming.

Design principles that recur

- **Fail soft, always.** A raised exception is scored as an outright miss, so the reranker, the dense route, the phrase routes and the model client each carry their own fault isolation. A component failure costs its own contribution, never the session.
- **Secondary routes stay light.** Every measurement agrees: over-weighting *any* secondary route costs HitRate. Category at 0.6, dense at 0.5, or phrases at 1.0 each drop $0.975 \rightarrow 0.970$. All shipped weights sit below that cliff.

- **Filters bind early routes only.** Hard constraints apply to routes 1–2; the rest bypass them, so a mis-scraped colour cannot hide the target. This is why a known first-write-wins bug in the constraint merge stopped mattering without ever being fixed.
- **The two speakers are kept apart.** Simulator handling and human handling share state but not code paths, and an automated check enforces that the human path executes zero times while scoring.

SECTION 07

What we learned about the game itself

The evaluator is participant-visible. Reading it produced more score than any algorithm we wrote.

- **The customer only tells you what you ask for.** A disclosure is released only when the question's attribute matches the constraint's classification. Ask the wrong thing and you learn nothing; re-ask a spent one and you get *"I don't have an additional preference for X."*
- **Asking nothing wastes the turn outright.** A null question returns *"ask me about one specific attribute"* and reveals nothing. Always ask something.
- **other** is the only question that cannot whiff. It matches any undisclosed constraint and returns up to two per turn. Simulator-specific, and our largest disclosed risk.
- **Two near-identical sentences mean opposite things.** *"an additional preference"* means the attribute is empty — retire it. *"a preference"* is the boundary customer deferring once — do **not** retire it, they answer normally afterwards.
- **Disclosures are near-verbatim target text.** Intent cards are built from the target's own **features** and **details**, so getting the customer to speak is getting them to quote the answer. This single property is why coverage-and-phrase reranking works so well, and why field-factor parity was the right call on mechanism alone.
- **The customer knows at most four things and is drained by turn 3.** The structural ceiling that closes most of the search space.

- **The first hit ends the session and freezes the rank.** The basis of the whole disclosure gate. Surfacing the target early at a bad rank is a *cost*, not a win.
- **Intent-override sessions cannot convert early.** Hits before the override fires on turn 3 or 4 are discarded, so ranking the target 1st on turn 1 scores nothing there.
- **Runs are deterministic.** A changed score means a changed agent — never run-to-run noise. Every claim in this document depends on that.

THE STANDING CAUTION

Tune to the **mechanism**, not to the strings. The specification says the organiser may add natural-language paraphrasing, and the hidden set is four times our size. Matching literal simulator phrases will not survive that; modelling "ask a targeted question, absorb the answer into state" will. This is why several features shipped flat — they are insurance against a dataset we cannot measure.

SECTION 08

Dead ends and negative results

Recorded deliberately. A regression written up in two minutes stops a teammate re-attempting the same idea on the final day, and there is no penalty here for a documented dead end.

IDEA	WHY IT SEEMED RIGHT	WHAT ACTUALLY HAPPENED
Blend search order into the reranker	Reranking should correct, not replace	Cost 0.03–0.04 at every nonzero weight. Removed.
Blend dense similarity into the reranker	A semantic signal is new information	Regressed monotonically from the first nonzero weight; 0.1 cost a session outright. It is a smoothed version of statistics coverage already reads. Ships disabled.
Conjunction route	A phrase too common to lead can still be decisive in combination — and it did narrow 50,000 to 100	Rescued none of the three sessions it was built for and lost another. Fusion scores an item by its <i>rank within a route</i> , not by how small the route is: the target sat ~50th of 100 near-identical robes. A small candidate set is only worth something if you can order it.
Honouring the retraction	The literal reading of "ignore my earlier preference"	–0.003875. The withdrawn text is a true attribute of the target, so deleting it deletes evidence.
Dropping retracted claims from phrase routes only	Keeps the reranking benefit, stops the pool widening	Flat and byte-identical — <i>and not inert</i> . Discarded 19 identifying routes across 16 sessions, several unique to one product. Flat on the public set is not the same as safe.
Personalization from the user profile	"May be used for personalization" in the starter kit	Measured across all 200 sessions: the profile is degenerate . Two fields are constant in all 200; one is derived from others; the only varying field appears in over half the sessions, so it cannot separate one target from 50,000. Zero upside.
Deferring disclosure further	If one turn of delay buys rank, more should buy more	Provably worthless — after turn 3 there is no new information in the universe. Every tighter variant is net negative.
Suppressing model reasoning traces	Free reasoning models waste the token budget on traces	Three switches tried live: disabling reasoning nulled the content outright ; excluding it only hid the trace without

IDEA	WHY IT SEEMED RIGHT	WHAT ACTUALLY HAPPENED
		freeing the budget. Plain payload scored best. Nothing provider-specific is sent.
The echo stub	A stub returning known words should be a safe test of the expansion route	Every word it returned was stripped as non-novel, so no route was ever appended and the run came back byte-identical. That looked like proof of safety and was proof of nothing.
Requiring "men" as a hard filter	The obvious fix for gendered results	5,900 products carry "men" outside the title as keyword spam, so women's listings satisfy it. Women's items still returned at ranks 1, 2 and 5.
"Turns are wasted once evidence runs dry"	Listed as a known gap for three features	Never true — no session runs past turn 4 except the misses. It is now deliberately false in the <i>other</i> direction: feature 05 spends those turns on purpose.

SECTION 09

How we keep ourselves honest

The measurement discipline is itself a deliverable — several of the findings above exist only because it was in place.

The rules

- **Definition of done.** A feature is not done until the evaluator has been re-run and the score movement written down: implement, re-run, diff, write the feature doc with the aggregate *and* all four per-scenario tables, then commit code and results snapshot together.
- **The score ratchet.** Runs all 200 sessions and **exits non-zero if the score fell**. TechnicalScore may rise or stay level, never fall. Critically, it separates *byte-identical* — the only honest way to claim "no effect on scoring" — from merely *score-equal*, where offsetting movements can hide a regression the hidden set would not forgive.
- **Two verification harnesses**, neither needing network or credentials: 90 feature, contract and isolation checks; 96 model, environment and breaker checks with

HTTP stubbed. Both are the pre-submission gate. Two robustness checks report as **XFAIL** rather than failures — a recorded decision, not a regression.

- **The isolation invariant.** A full run makes 566 simulator-path calls and **zero** human-path calls, asserted automatically — which is what keeps "the human path cannot affect the score" true as the patterns evolve, rather than letting it decay into folklore.
- **Sweeping properly.** The sweep tool builds one agent and reuses it across variants, so 30 variants take minutes rather than an hour — and it **aborts if the control arm does not reproduce the score of record exactly**, so a broken harness cannot quietly produce plausible numbers.

The habits that produced the best findings

- **Instrument before theorising.** Feature 06 was scoped as "dense retrieval" and would have built an embedding stack that could not have helped. An hour of instrumentation found three real bugs.
- **Sweep with a control arm.** Every ablation includes the shipped configuration as a row.
- **Re-sweep after a pipeline change.** Feature 10 changed the answer to a sweep run in feature 09: one parameter went from "+0.0001, indistinguishable" to "-0.0031, clearly wrong". The axes are not independent.
- **Write down what did not work.** Section 8 exists because of this rule.
- **Don't overrule the noise floor because the arrow is green.** Features 06 and 10 were both flagged as flat and shipped anyway on explicit mechanism arguments, stated as such.

SECTION 10

Feasibility disclosure

Required by the submission rules. These are not part of TechnicalScore.

Latency

STAGE	TIME
Agent construction — search index + embeddings	~13.5 s, once at startup
Per turn — mean	~55 ms
Per turn — median	~44 ms
Per turn — 95th percentile	~130 ms
Per turn — worst	240–400 ms
Full 200-session run, end to end	~35 s

Unlike the score, these are **not deterministic** — they move with machine load. Stable to within ~3 ms on the mean across three consecutive runs, but the worst single turn ranged 240–400 ms. Quote them as *typical*, not exact, and note they are single-machine numbers from a Windows development box.

Tokens and cost

ITEM	VALUE IN THE JUDGED CONFIGURATION
Language model	None — no call is made
Network access at runtime	None
API keys required	None
Estimated model cost	\$0.00
Reported token usage	0 prompt, 0 completion

Zeros are reported rather than the field being omitted, so the disclosure reads “we used no tokens” rather than “they didn’t say” — and a tool fails loudly if that ever stops being true. With the optional model enabled (not the judged configuration): OpenRouter free tier, \$0.00, 50 requests/day shared across models, ~1.5 s typical added latency, real token counts reported.

Dependencies

`numpy`, `scipy` and `scikit-learn` — added for the dense route, and the project's only third-party dependencies. Everything else is standard library, the model client included.

ONE HONEST CAVEAT

Runtime needs no network, and that claim is correct. But installing the dependencies does. If the judging environment is isolated end to end they must be pre-provisioned — and the failure mode is a **quiet wrong number**, **not a crash**: without them the agent silently degrades to sparse-only retrieval and scores **0.909858** instead of 0.912205.

SECTION 11

Where it stands, and what is left

One open lever, a measured ceiling, and four misses that no retriever can reach.

SCENARIO	N	HITRATE@10	MRR
boundary	10	1.000	1.000
browsing	80	0.9875	0.853
buying	80	0.975	0.852
intent_override	30	0.9667	0.880

The one open lever

The coverage score measures recall with no length normalization. It asks *how much of the customer's evidence is in this product* and never the converse — *how much of this product is the customer's evidence*. So a sprawling listing containing "100% Cotton" and "Button closure" among forty other features scores **identically** to a focused listing where those two things are the whole product.

That is precisely the failure shape of the 34 sessions that hit at ranks 2–8. Adding a precision term, or normalizing by document length, is principled and has been tried nowhere in the project. **It is the only remaining idea with a real mechanism**

behind it rather than a hyperparameter nudge. Honest expectation: it moves a handful of the 34, not all of them — roughly +0.005 to +0.015 if it works, flat if it doesn't.

The realistic ceiling

POOL	WORTH
34 hits at rank 2–8 promoted to rank 1	+0.0348
4 remaining misses converted to finds	+0.0160
Realistic ceiling	~0.947 vs 0.912205 today

The four remaining misses are unreachable — verified, not assumed

Their disclosed constraints don't discriminate at all. One discloses only "cotton" (9,775 products), "100% Cotton" (3,770), "Imported" (15,300) and "Button closure" (2,391). **Nothing lexical or semantic separates a target from 3,000 items when the evidence is identical across all of them.** A conjunction route that narrowed the set to 100 still could not order it; one miss was re-checked directly and sits at rank 15, moving only to 14 with all filters removed. Do not spend remaining time here.

Known gaps, in priority order

1. **The coverage precision term** — the one open lever, above.
2. **Four unreachable misses** — and a fifth is a **marginal** hit at rank 10 on turn 5, one position from being a miss again. HitRate 0.98 should be read as 0.975 plus a session hanging on the boundary of the cut, not as a robust figure.
3. **Turns 1–2 return a single recommendation** — contract-legal, never costs a find, but thin UX that reads oddly in a live demo. Disclosed here rather than left for a judge to discover.
4. **No broad exception guard on the response path.** Fault isolation exists at the inner layers, but an exception in the dialog-state or early retrieval code would escape, and a raised exception scores as a miss. The public set never triggers this and the organiser's evaluator catches exceptions anyway — so it is **insurance against a stricter hidden harness, not a known loss.** The fix shape is recorded, so this is a decision rather than a discovery.

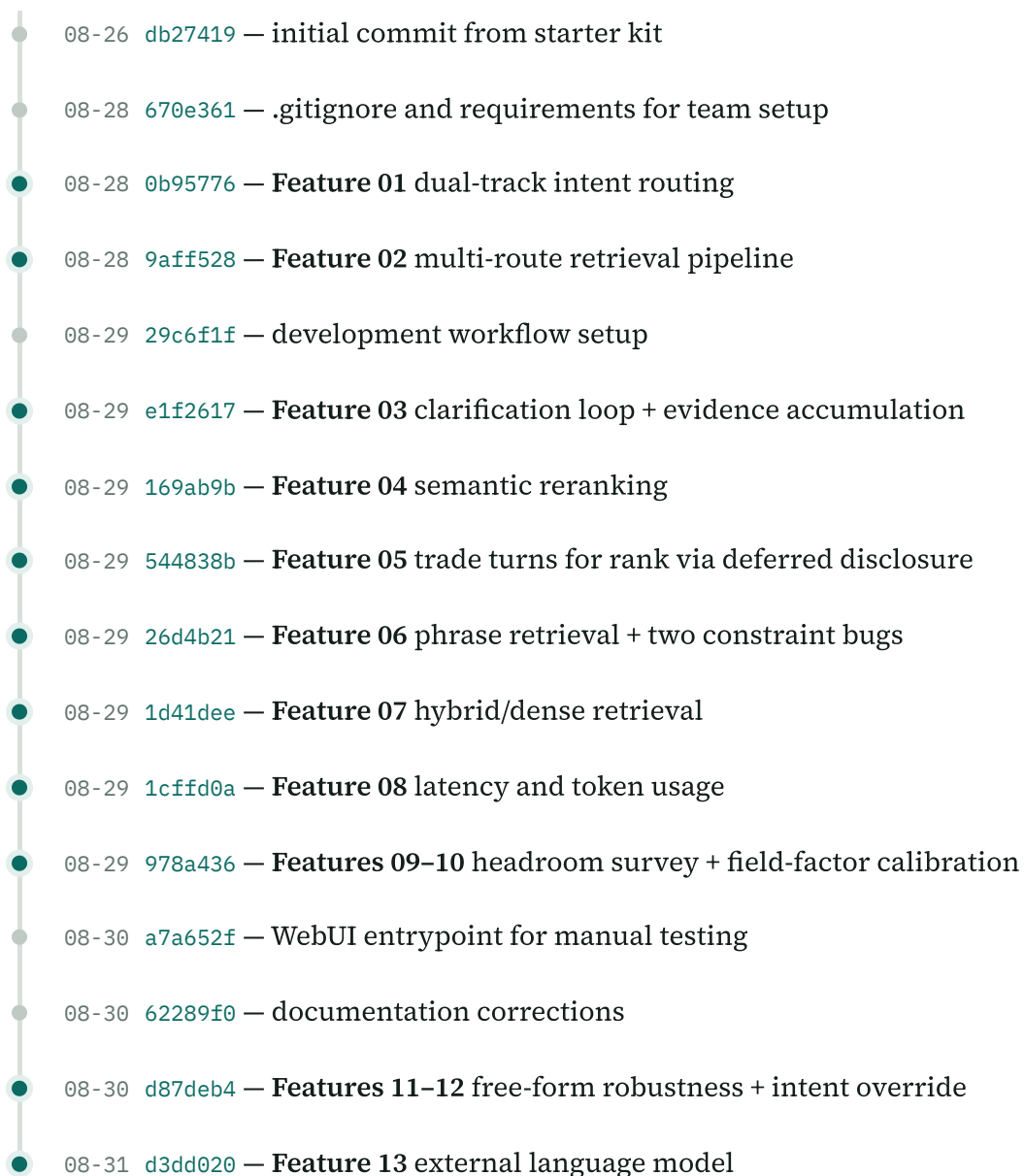
Explicitly closed

- **Personalization** — measured degenerate, zero upside.
- **Intent override** — 29/30 at 0.09 of a turn off its structural floor; both retraction routes measured and rejected.
- **Dialog strategy generally** — the four-constraint ceiling means no question policy can do better.

SECTION 12

Timeline

Twenty commits across six days on branch `dev` . `main` holds the untouched starter kit.



- 08-31 [f506810](#) — **Feature 14** model fixes, benchmarking, breaker, WebUI integration
- 08-31 [afa3168](#) — **Feature 15** free-form negation and slot-aware questioning
- 08-31 [18785da](#) — **Feature 16** generic attribute facets + score ratchet
- 08-31 [dff52e8](#) — README update

Running it yourself

```

pip install -r requirements.txt

py -m evaluator.local_evaluator                # full 200-session run
py tools/score_delta.py <before>.json <after>.json # markdown delta table
py tools/score_ratchet.py                      # REFUSES a change that lowers
the score
py tools/verify_features.py                    # 90 feature/contract/isolation
checks
py tools/verify_llm.py                         # 96 model/env/breaker checks,
no key needed
py tools/sweep_constants.py --axis A B         # coordinate-descent constant
sweep
py tools/feasibility_report.py                 # latency / token / cost tables
py -m webui.server                           # browser UI for manual testing

```

Use `py`, not `python3` — on the development Windows machine `python` and `python3` resolve to the Microsoft Store stub and fail. The organiser's README says `python3` because it assumes Linux, so our submission instructions cover both. The catalogue must be present at `data/catalog.jsonl` (50,000 rows) before the evaluator will run.

Every figure in this document is drawn from a committed results snapshot or a recorded measurement in `docs/features/`. Where a number is below the noise floor, it is labelled as such. Where a claim is unmeasured, it says so.

An editable Markdown copy of this record lives at `docs/PROJECT_HISTORY.md`.